React

The library for web and native user interfaces

Learn React API Reference

Introduction

Tentative Plan

- React/Vite/Typescript/Material UI/Programming Patterns
- Growing use of AI assistants in Software Dev
- “Effective AI Useage in Software Development”
  - Need to know Technologies, tools, patterns, styles that work for you
  - Need AI assistant to match that
- Skills-based AI assistance
  - Build knowledge/skills though initial labs/lectures
  - Start to construct a personal “Skills Repo” that AI can reference
  - Use AI to develop artifacts/code that you can review/maintain/understand
    - Not Vibe coding
    - Informed by Spec Driven Development
- Still forming assignment goals/assessment/deliverables for this!!!!

Effective AI Usage in Software Development

TypeScript Material UI

Need to know Technologies, tools, patterns that work

Initial Labs & Lectures

Build knowledge/skills though initial labs/lectures

Develop Skills Repo

Start to construct a personal "Skills Repo" that AI can reference

Skills-Based AI Assistance

Review & Maintain Code

Review artifacts/code generated by AI to ensure understanding

Generate Code & Artifacts

Start to construct a personal "Skills Repo" that AI can reference

Generate Code & Artifacts

Use AI to develop artifacts/ code that you can review/ maintain/understand

Personal Skills Repo

Your personal repository of referencable skills and knowledge

Knowledge & Skills Growth Over Time

Agenda

- Background.
- The V in MVC
- TSX (Typescript Extension Syntax).
- Developer tools..
- React Component basics.
- Material Design.

React

- **A Javascript framework for building dynamic Web** User Interfaces**.**
  - **A Single Page Apps technology.**
  - **Open-sourced in 2012.**

- **Client-side framework.**
  - **More a library than a framework.**

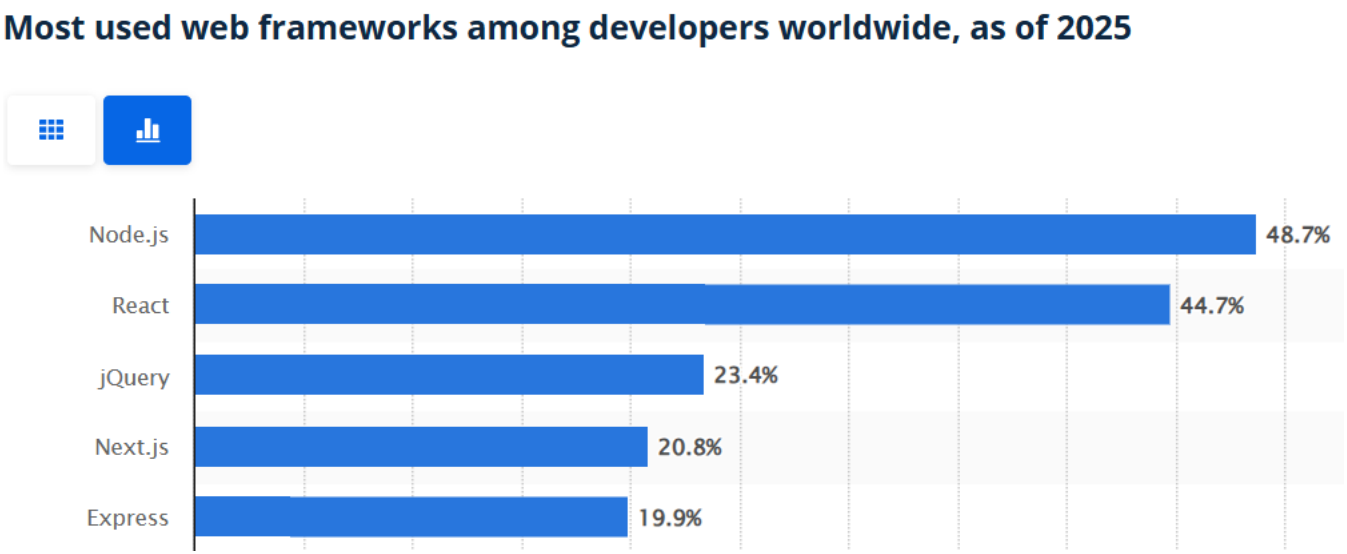
Why React?

AI Overview +6

React remains the top, most popular frontend library in 2025, with 44.7% adoption in the Stack Overflow survey and over 11 million websites using it. It consistently leads in popularity, developer desire, and job market demand over competitors like Vue, Angular, and Svelte. Gist +2

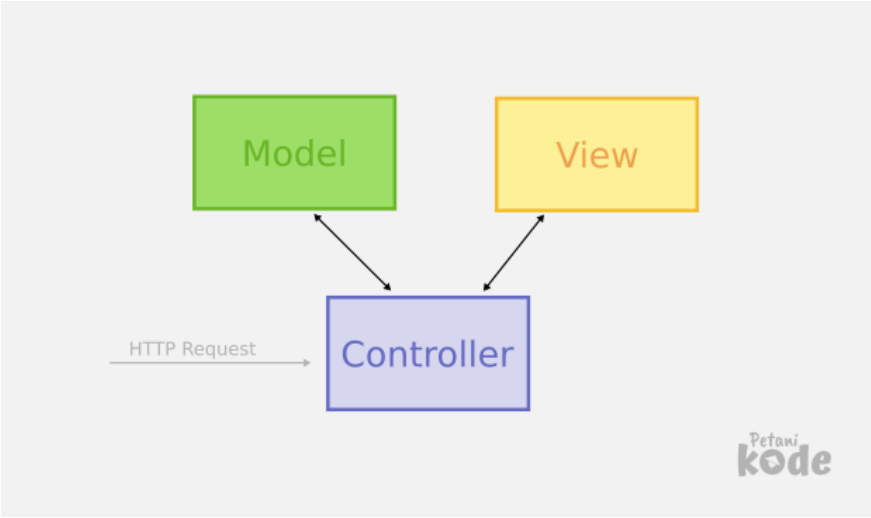
Key 2025 React Ecosystem Rankings & Trends:

- **Top Frameworks by Usage:** React is #1 (44.7%), followed by Angular (18.2%) and Vue.js (17.6%).
- **Most Desired/Admired:** Svelte leads in admiration (62.4%), but React holds the highest "desired" metric at 30.7%.



Before React

- MVC pattern **– The convention for** app design. **Promoted by market leaders**, e.g. **AngularJS (1.x), EmberJS, BackboneJS.**
- **React is not MVC, just V.**
  - **It challenged established best practice (MVC).**
- Templating widespread use in the V layer.
  - **React based on "components".**



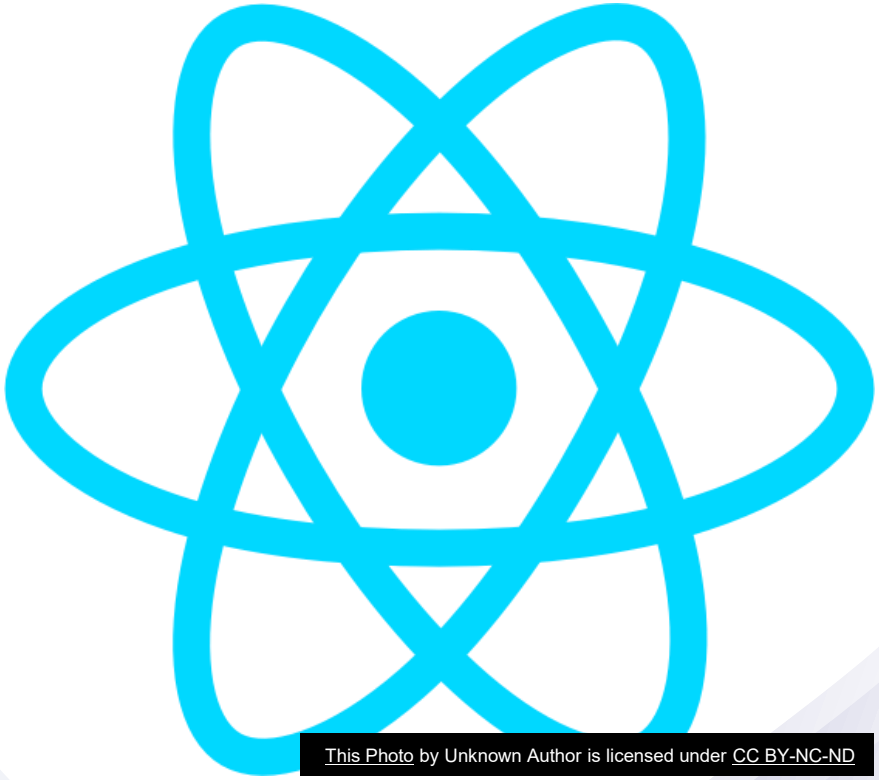
This Photo by Unknown Author is licensed under CC BY

React Components

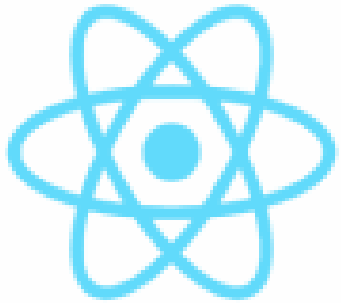
- **Philosophy:** *Build components, not templates.*
- **All about the User Interface (UI).**
  - Not focused on business logic or the data model (MVC)

- **Component - A unit comprised of:**

  UI description (HTML) + UI behaviour (JS)
  - **Two aspects are tightly coupled and co-located.**
    - Pre-React frameworks decoupled them.
  - **Benefits:**
    1. Improved Composition.
    2. Greater Reusability.



This Photo by Unknown Author is licensed under CC BY-NC-ND

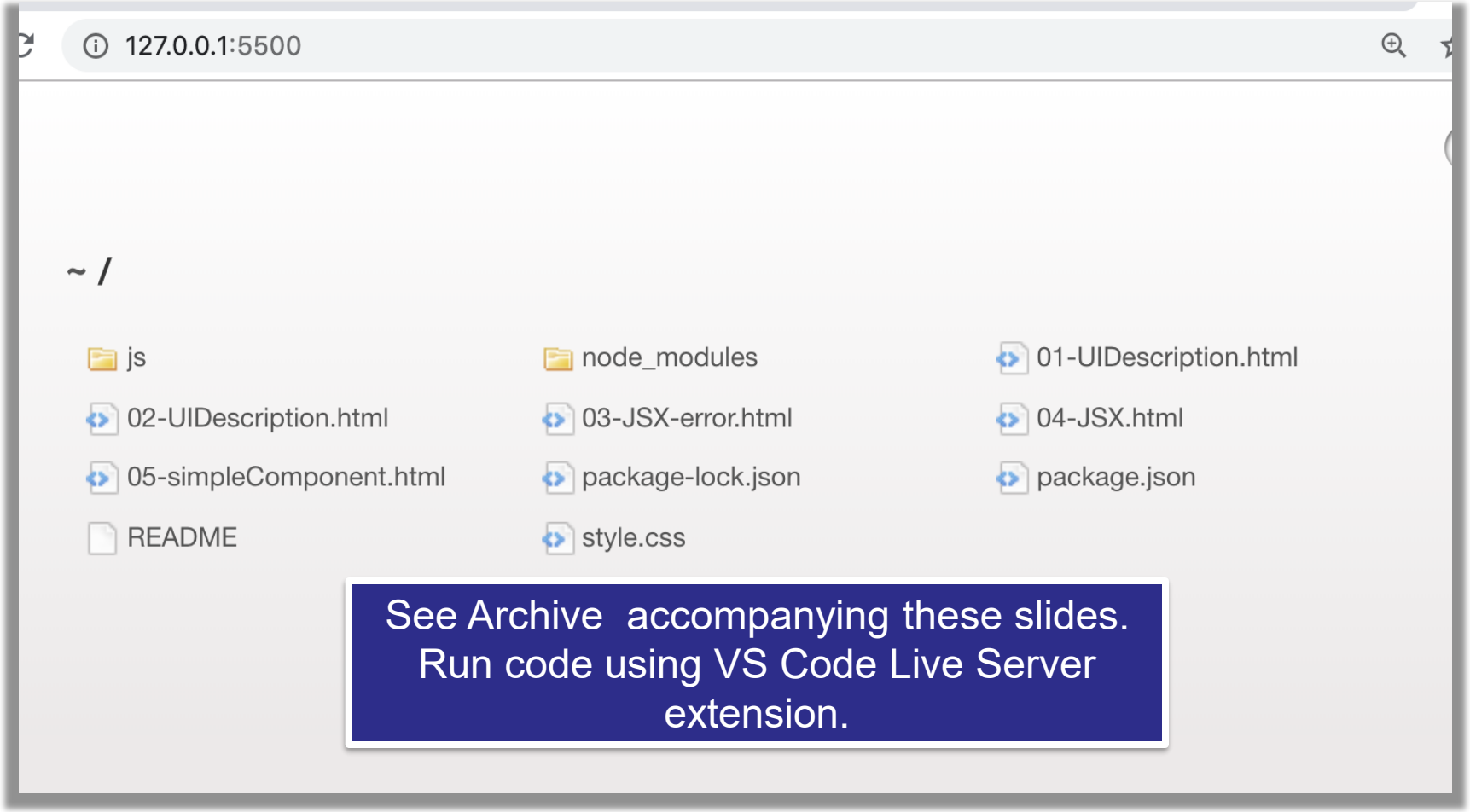
TypeScript  React

- Used to add type definitions to JavaScript codebases.
- TypeScript supports JSX (=>TSX)
- Include in your React project using **@types/react** and **@types/react-dom**
- Needs to be **Transpiled/Compiled** to Javascript to run in Browser/Client.

This Photo b Unknown Author is licensed under CC BY

In-Class Code Demos.

(See lecture archive)



Creating the UI Description.

(Vanilla React)

- React.createElement() **– creates a HTML element.**
- ReactDOM.createRoot() – **used to attach the created element to existing DOM node.**
- React.createElement() takes three or more **arguments**:

Type (e.g. h1, div, …)

Properties (style, event handler…)

Children(0 to many child elements)

```
<div id="mount-point"></div>
<script src="../node_modules/react/umd/react.development.js"></script>
<script src="../node_modules/react-dom/umd/react-dom.development.js"></script>
<script>
  // Create elements imperatively with React.createElement
  const header = React.createElement("h1", { className: "heading" }, "Languages");
```

  - **We don't use createElement() directly - too cumbersome. More later…**

- ReactDOM.createRoot () has 1 **argument:**

DOM element on which to render your React Root Element

```
// Render the elements
const rootElement = ReactDOM.createRoot(document.getElementById("mount-point"))
rootElement.render(root);
```

UI Description Implementation

(the imperative way)

- **See the demos:**
  - **Ref.** 01-UIDescription.html.

  - **Nesting** createElement**() calls (Ref**. 02-UIDescription.html)

- **Imperative programming** is a programming paradigm that uses statements that change a program's state.

  focuses on describing how a program operates, step by step.

- **Declarative programming** is a programming paradigm … that expresses the logic of a computation without describing its control flow.

  focuses on what the result should be without specifying how it should achieve the results

UI description implementation

(the declarative way)

- **TSX – TypeScript XML.**

- Declarative syntax for coding UI descriptions.

- Retains the full power of Typescript.
- Allows tight coupling between UI behavior and UI description.

- **However, must be transpiled before being sent to browser.**

- **Reference** 03-JSX-error.html and 04-JSX.html

```
const root = (
  <div className="pad">
    <h1 className="heading">Languages</h1>
    <ul>
      <li>Javascript</li>
      <li>Java</li>
      <li>Python</li>
    </ul>
  </div>
);
const rootElement =  ReactDOM.createRoot( document.getElementById("mount-point"
rootElement.render(root);
```

REPL (Read-Evaluate-Print-Loop) transpiler.

BABEL

Docs Setup Try it out Videos Blog Donate Team GitHub

SETTINGS

- [x] Evaluate
- [x] Line Wrap
- [x] Prettify
- [ ] File Size
- [ ] Time Travel

Source Type

Module

TARGETS

defaults, not ie 11, not ie_mob 11

PRESETS

- [x] react
- [ ] flow
- [x] typescript
- [ ] stage-3
- [ ] stage-2
- [ ] stage-1
- [x] stage-0

OPTIONS

```
const root = (
  <div className="pad">
    <h1 className="heading">Languages</h1>
    <ul>
      <li>Javascript</li>
      <li>Java</li>
      <li>Python</li>
    </ul>
  </div>
);
const rootElement =  ReactDOM.createRoot(
document.getElementById("mount-point"  ) )
rootElement.render(root);
```

```
const root = /*#__PURE__*/ React.createElement(
  "div",
  {
    className: "pad"
  },
  /*#__PURE__*/ React.createElement(
    "h1",
    {
      className: "heading"
    },
    "Languages"
  ),
  /*#__PURE__*/ React.createElement(
    "ul",
    null,
    /*#__PURE__*/ React.createElement("li", null,
"Javascript"),
    /*#__PURE__*/ React.createElement("li", null,
"Java"),
    /*#__PURE__*/ React.createElement("li", null,
"Python")
  )
);
const rootElement =
ReactDOM.createRoot(document.getElementById("mount-
point"));
rootElement.render(root);
```

Reference
04-JSX.html

TSX(TypeScript XML)

- **JSX(Javascript XML)** is a file syntax extension used by React. JSX resembles HTML, allowing developers to write UI components with an HTML-like (it is actually XML).
- **TSX is the TypeScript version of JSX,** TypeScript is a superset that adds static typing in JavaScript.
  - Typescript files containing TSX use the .tsx extension.
- Some minor HTML tag attributes differences, e.g. className (class), htmlFor (for).

```
const App: React.FC<AppProps> = ({ title }) => (
  <div className="app-container">
    <h1 className="app-header">{title}</h1>
    <button onClick={() => console.log('Button clicked!')}>
      Click me
    </button>
  </div>
);
```

- Allows UI description to be coded **in a declarative style** and be in-lined in TypeScript.

- **Combines templating with the power of TS.**

TSX Transpiling

- **So browsers don't do Typescript!**
  - Needs to be **Transpiled to Javascript**
- What can we use to Transpile?
  - The Babel platform.
  - The Vite library.

- How?
  1. Manually, via REPL environment or command line.
     - When experimenting only.
  2. Using an instrumented web server – Vite library instrumentation.
     - Ideal for development.
  3. Using bundler tools as part of the build process – Vite again.
     - Production standard.

React Components.

- **We develop COMPONENTS.**
  - A TS function that returns a UI description, i.e. TSX**.**

- **We reference a component like a <u>HTML tag.</u>**

  e.g.

```
const rootElement =
    ReactDOM.createRoot(document.getElementById("mount-point"));
rootElement.render( <DynamicLanguages/> );
```

- **Reference** 05-simpleComponent.html